

Embedded C Cheat Sheet

"A practical reference for bare-metal
and embedded systems programming."

by Matias Back

Frequently Used Headers

Context: These headers provide the most commonly used types, macros, and utility functions in embedded software.

<stdint.h> Fixed-width integer types (uint8_t, int32_t)
<stdbool.h> Boolean type (bool, true, false)
<stddef.h> NULL, size_t, offsetof()
<string.h> memcpy(), memset(), memcmp()
<assert.h> Runtime assertions
<limits.h> Integer limits and CHAR_BIT

Fixed-Width Integer Type

Context: Microcontrollers may use different CPU architectures where the size of int can vary. When working with hardware registers, communication protocols, binary data, or memory-mapped peripherals, the data size must be explicitly defined.

Examples:

uint8_t status; // 8-bit unsigned, flags, states, small values
uint16_t adc_value; // 16-bit unsigned: ADC readings, sensors
uint32_t timestamp; // 32-bit unsigned: timestamps, counters

Type Reference Table

Type	Bits	Bytes	Range
uint8_t	8	1	0 - 255
int8_t	8	1	-128 - 127
uint16_t	16	2	0 - 65,535
int16_t	16	2	-32,768 - 32,767
uint32_t	32	4	0 - 4,294,967,295
int32_t	32	4	-2,147,483,648 - 2,147,483,647

Useful pointer-sized types:

uintptr_t address;
intptr_t offset;

Important Keywords

Context: Essential keywords that dictate memory storage, optimization, and variable visibility in embedded systems.

volatile

Context: Forces fresh memory reads for data changed by hardware, ISRs, or DMA. Used for registers and shared variables. (Note: Does not grant atomicity or thread safety).

```
volatile uint8_t button_pressed;  
volatile uint32_t * const GPIOA_ODR = (volatile  
uint32_t*)0x40020014;
```

const

Context: Read-only data. Saves RAM by (usually) storing in Flash/ROM. Used for lookup tables and calibration. (Note: Flash placement is compiler-dependent).

```
const uint8_t lookup_table[4] = { 10, 20, 30, 40 };
```

static

Context: static is used either to preserve data between function calls or to restrict visibility to the current source file.

Inside a function. The variable keeps its value between calls:

```
void tick(void) {  
    static uint32_t counter = 0;  
    counter++;  
}
```

Outside a function. The variable can only be accessed within the current source file:

```
static uint32_t g_system_ticks;
```

Bitwise Operators

Context: Hardware peripherals are controlled through registers where individual bits enable, disable, or configure features.

Operator Name

&	AND
	OR
^	XOR
~	NOT
<<	Left Shift
>>	Right Shift

Combined:

&= mask, |= set, ^= toggle, <<= shift left, >>= shift right

Example:

```
uint8_t result = a & b; // Performs bitwise AND  
PORTA |= BIT(5); // Sets pin 5 HIGH to light up the LED
```

Useful Macros

Context: Utility macros simplify repetitive code at compile time with "zero" runtime cost.

BIT(n) creates a bitmask for safely changing or checking specific bits in a register.

ARRAY_SIZE(x) automatically counts the number of elements in an array using its memory size.

Definitions:

```
#define BIT(n) (1U << (n))  
#define ARRAY_SIZE(x) (sizeof(x) / sizeof((x)[0]))
```

Examples:

```
REG |= BIT(3);  
size_t len = ARRAY_SIZE(buffer);
```